

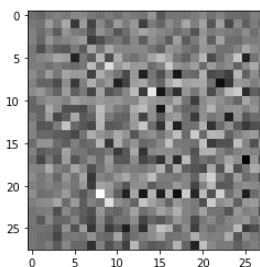
Assignment 6- GAN model

Part 1- Number generator

For the first part of this lab, we want to use GAN to create hand written numbers from 0 to 9. For this propose, we will use tensorflow to create the model from scratch. Nowadays, they are many already developed models, which can create these digits but our goal is learning so for this section, we will create it ourselves.

a) From Keras, import the mnist dataset. We need to reshape the images and normalize them.

b) We can now create our generator model. We use Conv2DTranspose (upsampling) layers to produce the image from seed (Generator part). Call this generator model without training just to see what happens when we import a noise to model.



The result does not look nice but it is fine. The generator is not trained yet.

c) Now, create discriminator model using Keras. Like what we did above, apply this discriminative model on top of the above generate image and see the result. Maybe looking at the output does not reveal enough information. Instead, we can apply a cross_entropy function to compare the output with the real one.

d) We need to create two loss functions.

- Discriminator loss: This method quantifies how well the discriminator is able to distinguish real images from fakes. It compares the discriminator's predictions on real images to an array of 1s, and the discriminator's predictions on fake (generated) images to an array of 0s.

- Generator loss: The generator's loss quantifies how well it was able to trick the discriminator. Intuitively, if the generator is performing well, the discriminator will classify the fake images as real (or 1). Here, we will compare the discriminators decisions on the generated images to an array of 1s.

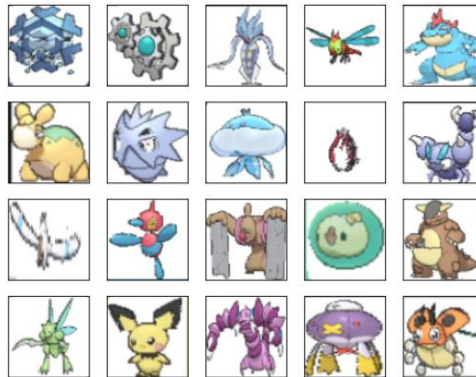
e) Now, is the time for training the model. It is recommended to take a checkpoint on model. The training loop begins with generator receiving a random seed as input. That seed is used to produce an image. The discriminator is then used to classify real images (drawn from the training set) and fakes images (produced by the generator). The loss is calculated for each of these models, and the gradients are used to update the generator and discriminator. There is a function provided for you to train the model and save the image.

You can call the train function with two elements train_dataset and number of epochs. The model takes a while to train but saving checkpoints allows us to get back to model whenever it is needed. While the model is loading, pay attention to the changes in epochs and results. After done with training, take a checkpoint and save the model. I recommend to download the model on your Google drive or a local machine. This helps not to re-train the model next time that you need to retrieve the model.

I also provided a code to save the steps of changes as a gif file.

Next example is inspired from: [Intro to Generative Adversarial Networks \(GANs\) | Towards Data Science](#) and [d2l.ai](#). The goal of this model is to create the characters of pokemon using a GAN model. The dataset that we use is from d2l package ([d2l · PyPI](#)).

b) As we discussed earlier, it is always good to visualize our data to get better insights. Run the provided code to see the images.



Now, call the `G_block` package and use 20 as the number of layers (input of the class). To see the dimension of the output, you can use:

d) Now is the time to create discriminator. The block is already provided for you. Import a signal in the form of zeros into the model. This input has stride of 2, in a 16x16 image and for three channels which means `x = tf.zeros((2, 16, 16, 3))`

```
d blk(x).shape
```

Part 3- Modern AI tools for generating

A different approach has become very important in modern generative AI: diffusion models.

Diffusion models also begin with random noise, but they generate images differently. Instead of asking a generator to create the complete image in one step, a diffusion model gradually removes noise over a sequence of steps until an image emerges.

GAN: Random noise → Generator → Generated image

Diffusion: Random noise → Denoise → Denoise → Denoise → ... → Generated image

Another major difference is that a diffusion model does not require a discriminator during image generation. It learns how to reverse a noise-adding process. Modern systems such as Stable Diffusion combine diffusion with text embeddings, allowing us to describe the image we want using natural language. Stable Diffusion performs the diffusion process in a compressed latent space, which reduces the computational cost compared with operating directly on image pixels.

Follow this diffusion code to generate an image using the provided prompt.

```
import torch
from diffusers import StableDiffusionPipeline
import matplotlib.pyplot as plt

model_id = "stable-diffusion-v1-5/stable-diffusion-v1-5"

pipe = StableDiffusionPipeline.from_pretrained(
    model_id, torch_dtype=torch.float16, use_safetensors=True)

pipe = pipe.to("cuda")

prompt = "a futuristic city at sunset, highly detailed photograph"

image = pipe(prompt, num_inference_steps=25).images[0]

plt.figure(figsize=(7,7))
plt.imshow(image)
plt.axis("off")
plt.show()
```

Compare the generated images and execution time when using 5, 25, and 50 inference steps. What trade-off do you observe?